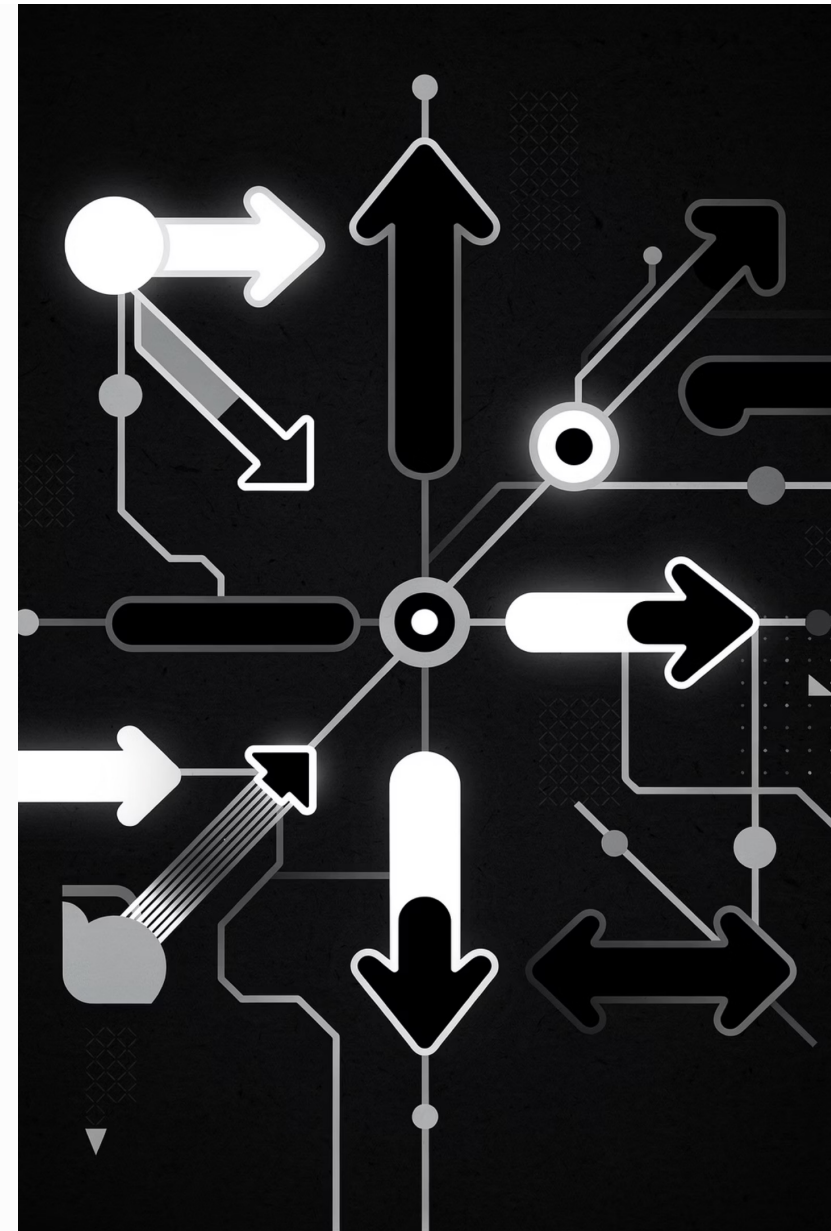


מצביעים חכמים — Smart Pointers

אובייקטים שהשימוש בהם דומה למצביעים רגילים, אך עם יתרונות משמעותיים: שליטה על בנייה, הריסה, העתקה והתייחסות.



הבעיה

ניהול זכרון ב-C++ →

- In C++, we allocate memory with `new`... but we must remember to `delete` it manually.
- Sounds simple... until reality happens:
 - Functions exit early
 - Exceptions are thrown
 - Multiple owners appear
 - Someone simply forgets

❌ Memory Leaks

Memory is allocated... but never freed

❌ Dangling Pointers

Memory is freed... but still accessed

❌ Double Delete

Same memory freed twice → undefined behavior

דוגמא →

בעיות →

```
void process() {  
    int* data = new int(42);  
    if (someCondition()) {  
        return; // 💣 Memory leak – delete is never called  
    }  
    delete data;  
}
```



מהו מצביע חכם?

המטרה היא ליצור אובייקטים שהשימוש בהם יהיה דומה למצביעים רגילים, אך עם התנהגות מבוקרת ומורחבת.

בנייה והריסה

ערך ברירת מחדל 0 בבנייה; האחריות למחיקת השטח המוצבע עוברת למצביע עצמו — מניעת דליפות זיכרון.

העתקות

שליטה מלאה: האם להעתיק את השטח המוצבע? את המצביע עצמו? או למנוע העתקות כלל?

התייחסות

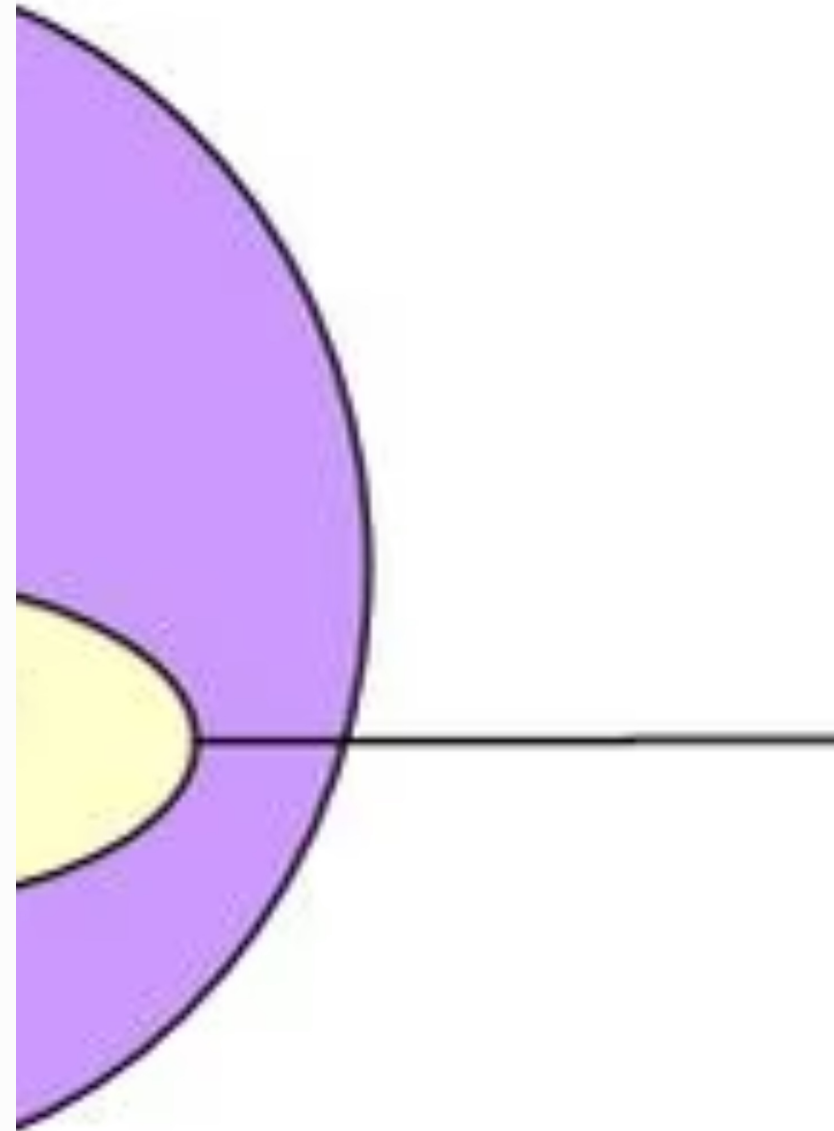
מה קורה כשמתייחסים לאובייקט דרך המצביע החכם? אנו קובעים זאת.

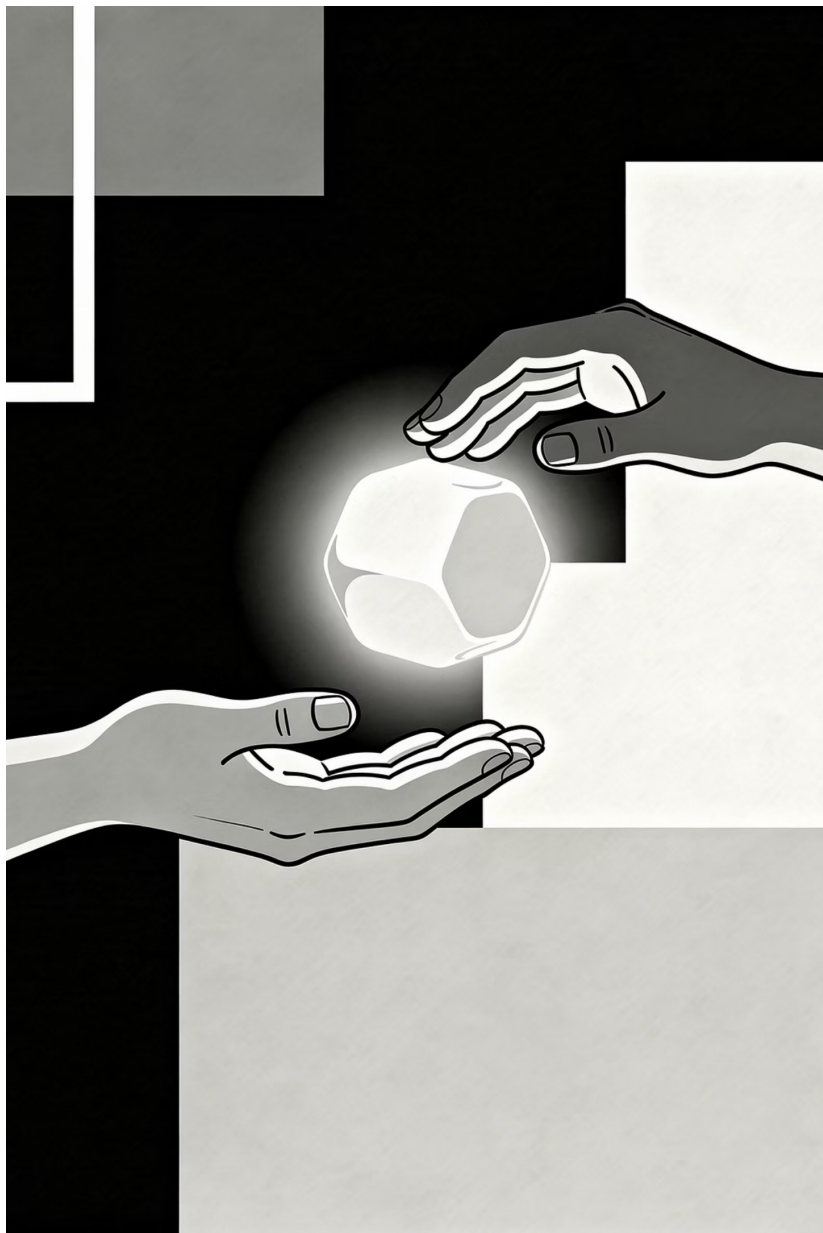
מבנה מחלקת SmartPtr

כדי להכריח את המשתמש להגדיר את הטיפוס המוצבע, יוצרים את המחלקה באמצעות תבנית: (template)

```
template<class T>
class SmartPtr
{
public:
    SmartPtr(T* realPtr = 0);
    ~SmartPtr();
    SmartPtr(const SmartPtr& rhs);
    SmartPtr& operator=(const SmartPtr& rhs);
    T* operator->() const;
    T& operator*() const;
private:
    T *pointee;
};

SmartPointer<Rational> pr = new Rational(2,5);
cout<<pr->getNumerator()<<endl;
```





בנייה, השמה והריסה — שאלת הבעלות

אם (ורק אם) מצביע חכם הוא הבעלים של האובייקט עליו הוא מצביע, הוא אחראי למחיקתו בעת הריסתו.

מימוש בסיסי — auto_ptr

```
template<class T>
class auto_ptr
{
public:
    auto_ptr(T *ptr = 0): pointee(ptr) {}
    ~auto_ptr() { delete pointee; }
    ...
private:
    T *pointee;
};
```

מניעת העתקה

פתרון מגביל — לא תמיד מתאים.

העתקה עמוקה

הקצאת אובייקט חדש זהה בערכו — יקרה, וגם לא תמיד ידוע איזה טיפוס ליצור - אולי יורש מ- T ?

בעיית העתקה פשוטה

העתקת המצביע בלבד תיצור שני מצביעים חכמים המצביעים לאותו אובייקט — שניהם ינסו למחוק אותו בהריסה. תוצאה מצערת.

העברת בעלות — הפתרון הגמיש

בעת העתקה או השמה, הבעלות על האובייקט המוצבע מועברת. לכל אובייקט יש מקסימום בעלים אחד.

```
template<class T>
auto_ptr<T>::auto_ptr(auto_ptr<T>& rhs)
{
    pointee = rhs.pointee;    // transfer ownership
    rhs.pointee = 0;          // rhs no longer owns anything
}

template<class T>
auto_ptr<T>& auto_ptr<T>::operator=(auto_ptr<T>& rhs)
{
    if (this == &rhs) return *this;
    delete pointee;
    pointee = rhs.pointee;
    rhs.pointee = 0;
    return *this;
}
```



שימו לב: הפרמטר אינו — **const** כי אנו משנים את המקור ממנו מעתיקים. זה דבר לא רגיל!

סכנת העברה כערך

בשל העובדה כי העתקה גוררת העברת בעלות, העברת מצביע חכם כערך היא מעשה שבדרך כלל מצטערים עליו:

```
// this function will often lead to disaster
void printListNode(ostream& s, auto_ptr<ListNode> p)
{
    s << *p;
}
// After call: ptn points to NULL!
```

העברה כייחוס 

`const auto_ptr<ListNode>& p` — `p` — אינו אובייקט, אין אתחול, הבעלות
נשמרת. גם יעיל יותר.

העברה כערך 

הבעלות עוברת לפרמטר `p`. עם סיום הפונקציה, האובייקט נהרס ו-`ptn` מצביע ל-`NULL`.

הריסת מצביע חכם

```
template<class T>
SmartPointer<T>::~SmartPointer()
{
    if (*this owns *pointee)
    {
        delete pointee;
    }
}

// In the auto_ptr implementation:
if (pointee)
{
    delete pointee;
}
```

לא תמיד יש צורך לבדוק לפני המחיקה — תלוי במימוש. אם כל מצביע חכם מצביע על אובייקט משלו, חייבים למחוק. אם הוא תמיד מצביע על אובייקט חיצוני — לעולם אין למחוק.

מימוש אופרטורי התייחסות (Dereference)

operator*

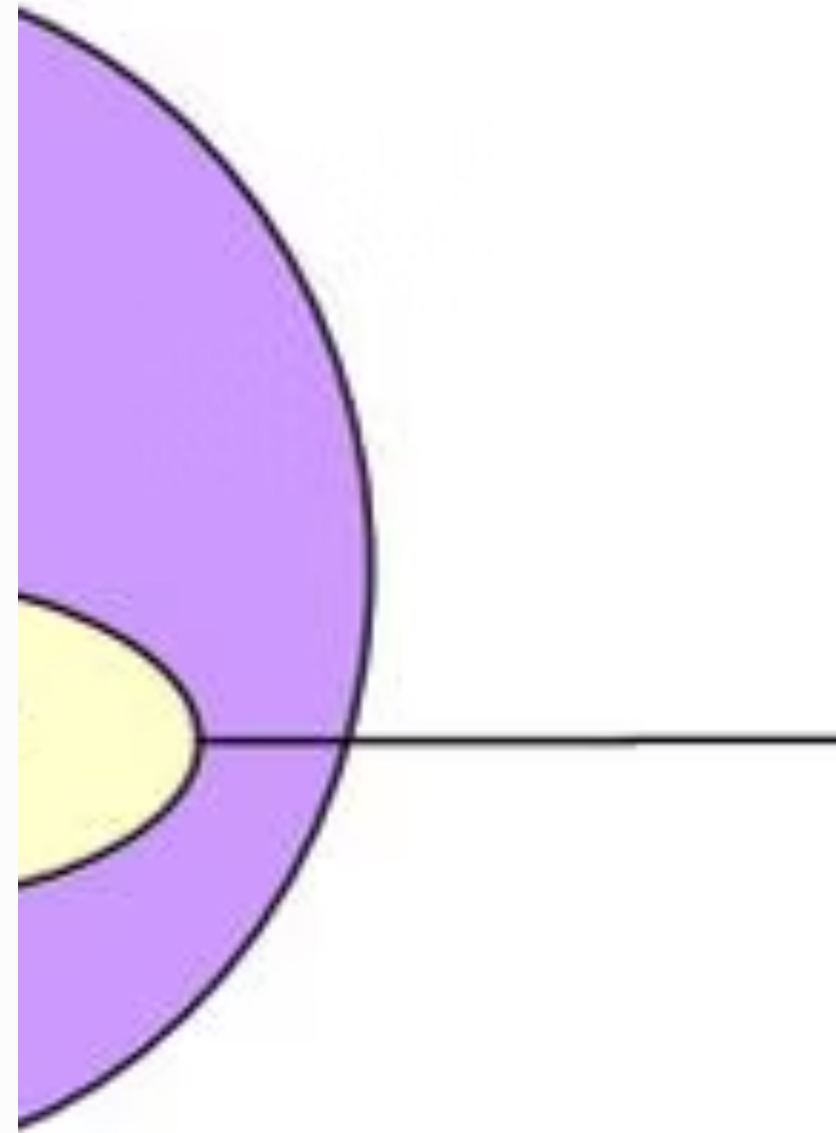
```
template<class T>
T& SmartPtr<T>::operator*()
const
{
    perform "smart pointer"
    processing;
    return *pointee;
}
```

מחזיר ייחוס — לא ערך! החזרת ערך תגרום לחיתוך
אובייקט (object slicing) ולכשל בפונקציות
וירטואליות.

operator->

```
template<class T>
T* SmartPtr<T>::operator->()
const
{
    perform "smart pointer"
    processing;
    return pointee;
}
```

מחזיר מצביע — מאפשר הפעלת פונקציות וירטואליות
בצורה הרצויה.



בדיקה לריקנות (Nullness)

כיצד בודקים אם מצביע חכם מצביע ל NULL? - ניסיון ישיר נכשל:

```
if (ptn == 0) ... // error!  
if (ptn) ... // error!  
if (!ptn) ... // error!
```

פשרה!() operator :

מחזיר true אם ורק אם המצביע הוא NULL. מאפשר שימוש חלקי נוח, ומגביל חלק מהסיכונים.

אפשרות 2 void*() operator :

אופרטור המרה מרומזת — מאפשר if(ptn) ו- if(ptn==0). אך יוצר בעיות: השוואה בין טיפוסים שונים לחלוטין עלולה לעבור קומפילציה!

אפשרות 1 isNull() :

פונקציית מחלקה המחזירה תוצאת השוואה ל NULL- פשוטה אך לא אינטואיטיבית.

המרה למצביעים רגילים

לעתים יש צורך להפעיל פונקציה המקבלת מצביע רגיל. ניתן להוסיף אופרטור המרה:

```
template<class T>
class SmartPtr
{
public:
    operator T*() { return pointee; }
    ...
};
update(pp); // now works!
```



בעיה עקרונית: פונקציית ההמרה מאפשרת לעקוף את המצביע החכם לחלוטין — כולל ביצוע `delete pt` על אובייקט, מה שיגרום למחיקה כפולה ולהתנהגות לא מוגדרת.

כלל המפתח להמרות

הכלל

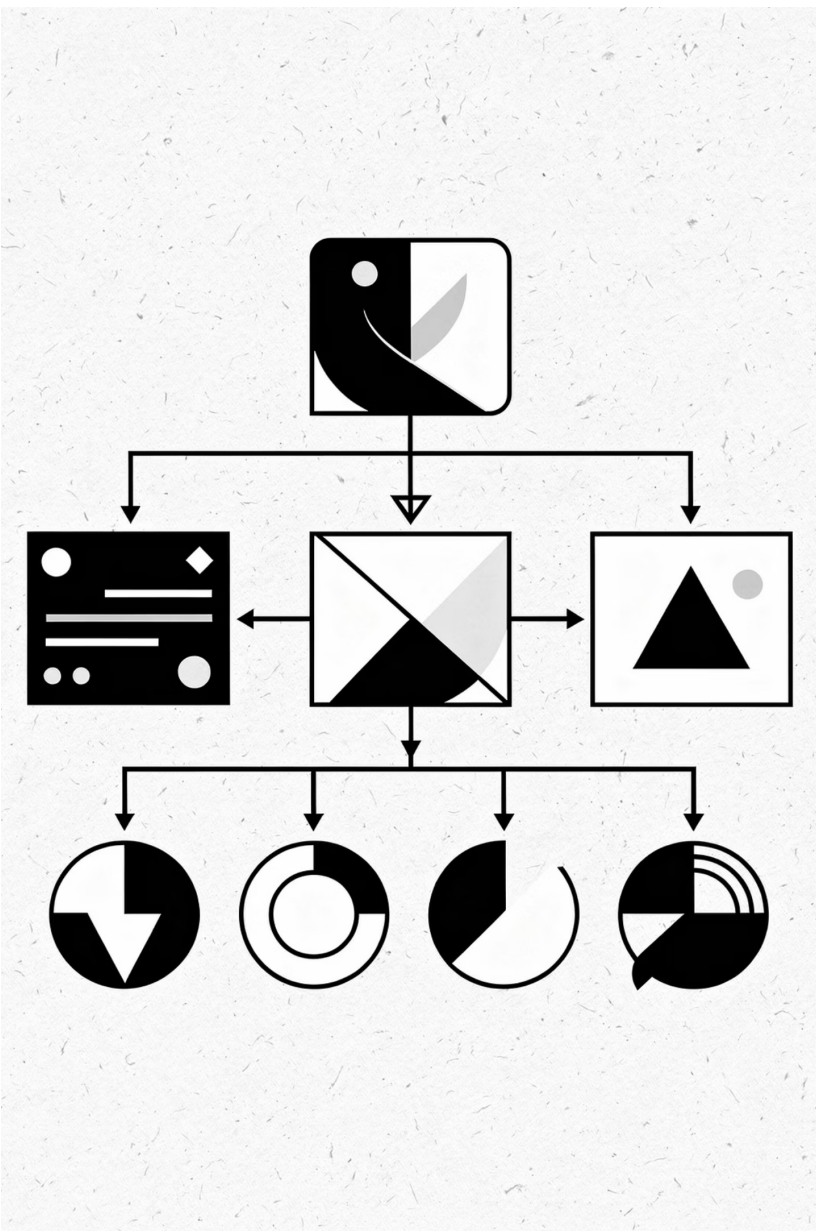
לא לספק אופרטורים להמרה בלתי מפורשת (implicit conversion) חכם למצביע רגיל — כל עוד אין סיבה טובה **ממש** טובה לכך.

הסיכון

ככל שהמצביע החכם דומה יותר לרגיל, המשתמשים עלולים לשכוח את ההבדל ולבצע — delete מה שיגרום למחיקה כפולה.

הגבלת הקומפיילר

הקומפיילר לא מרשה יותר מהמרה אחת מוגדרת על ידינו. לכן `SmartPointer<char>` לא יומר אוטומטית ל-`String` דרך `char*`.



מצביעים חכמים והמרה מבוססת ירושה

אחת הבעיות המורכבות ביותר: כיצד לאפשר המרות בין מצביעים חכמים המשקפות קשרי ירושה?

היררכיית MusicProduct

```
void displayAndPlay(const MusicProduct* pmp, int numTimes);
```

```
// Works fine with raw pointers:
```

```
Cassette *funMusic = new Cassette("Radio Ga Ga");
```

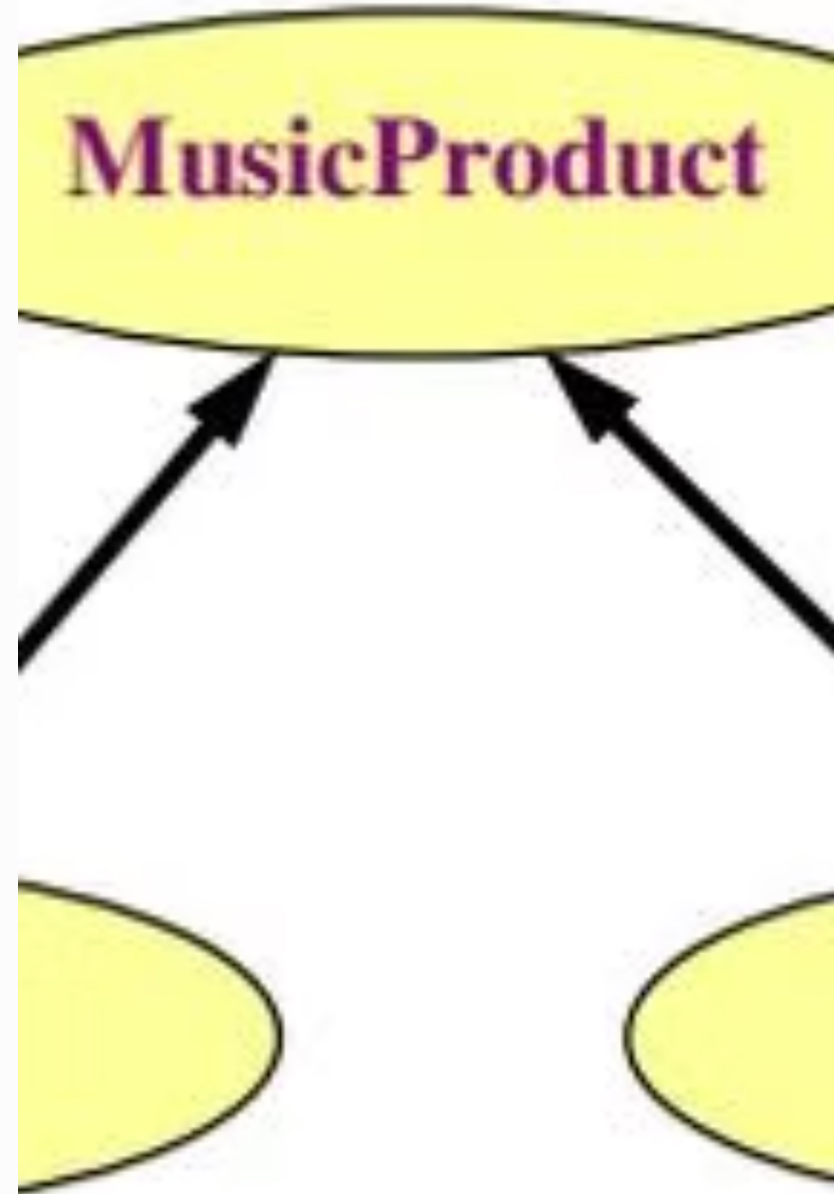
```
displayAndPlay(funMusic, 10); // OK
```

```
// Fails with smart pointers:
```

```
SmartPointer<Cassette> funMusic(new Cassette("Radio Ga Ga"));
```

```
displayAndPlay(funMusic, 10); // error!
```

הבעיה `SmartPointer<Cassette>` ו: `SmartPointer<MusicProduct>` – הם טיפוסים שונים לחלוטין – אין ביניהם קשר ירושה, למרות שנוצרו מאותה תבנית.



פתרון: אופרטורי המרה ספציפיים

```
class SmartPtr<Cassette>
{
public:
    operator SmartPtr<MusicProduct>()
    { return SmartPtr<MusicProduct>(pointee); }
    ...
};
```

הפתרון הגמיש: Member Templates

פונקציית המרה מבוססת תבנית בתוך המחלקה — הקומפיילר מחולל אוטומטית את ההמרה הנדרשת, בדיוק כמו שעובד עם מצביעים רגילים.

החיסרון הגדול

יש להגדיר אופרטורים ספציפיים לכל מקרה — סותר את כל רעיון התבנית (template). בהיררכיה עמוקה, יש לכתוב אופרטורים לכל ההיררכיה שמעל.

הפתרון האוטומטי — Member Templates

```
template<class T>
class SmartPtr
{
public:
    SmartPtr(T* realPtr = 0);
    T* operator->() const;
    T& operator*() const;

    template<class newType>
    operator SmartPtr<newType>()
    {
        return SmartPtr<newType>(pointee);
    }
    ...
};
```

הקומפיילר מחפש ומוצא את פונקציית התבנית, מחולל את ההמרה הנדרשת, ובודק אם ניתן לבנות `SmartPtr<MusicProduct>` ממצביע `Cassette*` — וכן ניתן, כי זו המרה חוקית בשפה.

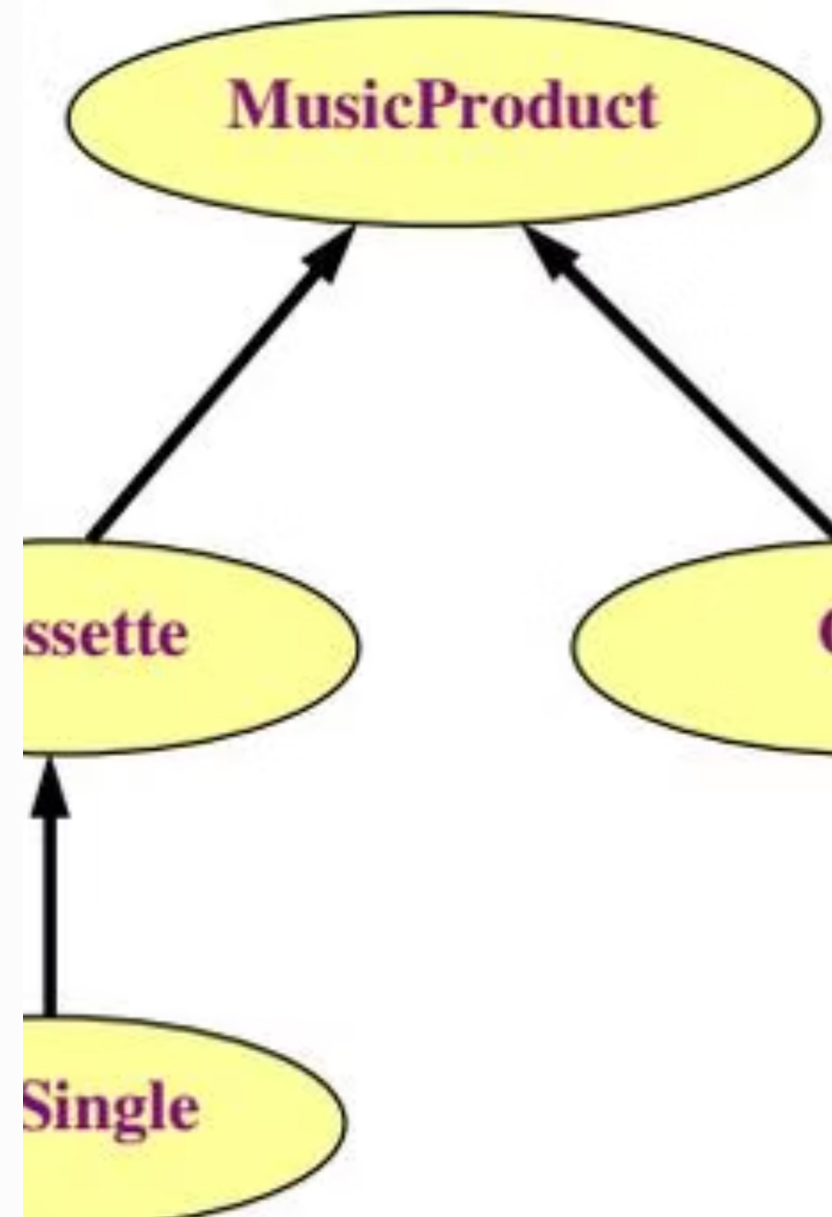


ההמרה תצליח אם ורק אם המצביעים הפנימיים ניתנים להמרה בשפה עצמה — שומרים על כל ההגבלות הנכונות.

מגבלת — Member Templates – 17- משמעות

```
void displayAndPlay(const SmartPtr<MusicProduct>& pmp, int  
howMany);  
void displayAndPlay(const SmartPtr<Cassette>& pc, int howMany);  
  
SmartPtr<CasSingle> dumbMusic(new CasSingle("Innuendo"));  
displayAndPlay(dumbMusic, 1); // error! ambiguous
```

עם מצביעים רגילים, ההמרה ל `*Cassette`-תועדף על פני `*MusicProduct` עם מצביעים חכמים, כל פונקציות ההמרה שקולות — הקריאה דו-משמעית.



סיכום חסרונות Member Templates

קשה לתחזוקה



השיטה אינה מספקת פתרון פשוט, קל להבנה ולתחזוקה.

דו-משמעות בירושה עמוקה

כשיש מספר overloads, הקומפיילר אינו מבדיל בין המרה "קרובה" ל"רחוקה" בהיררכיה.

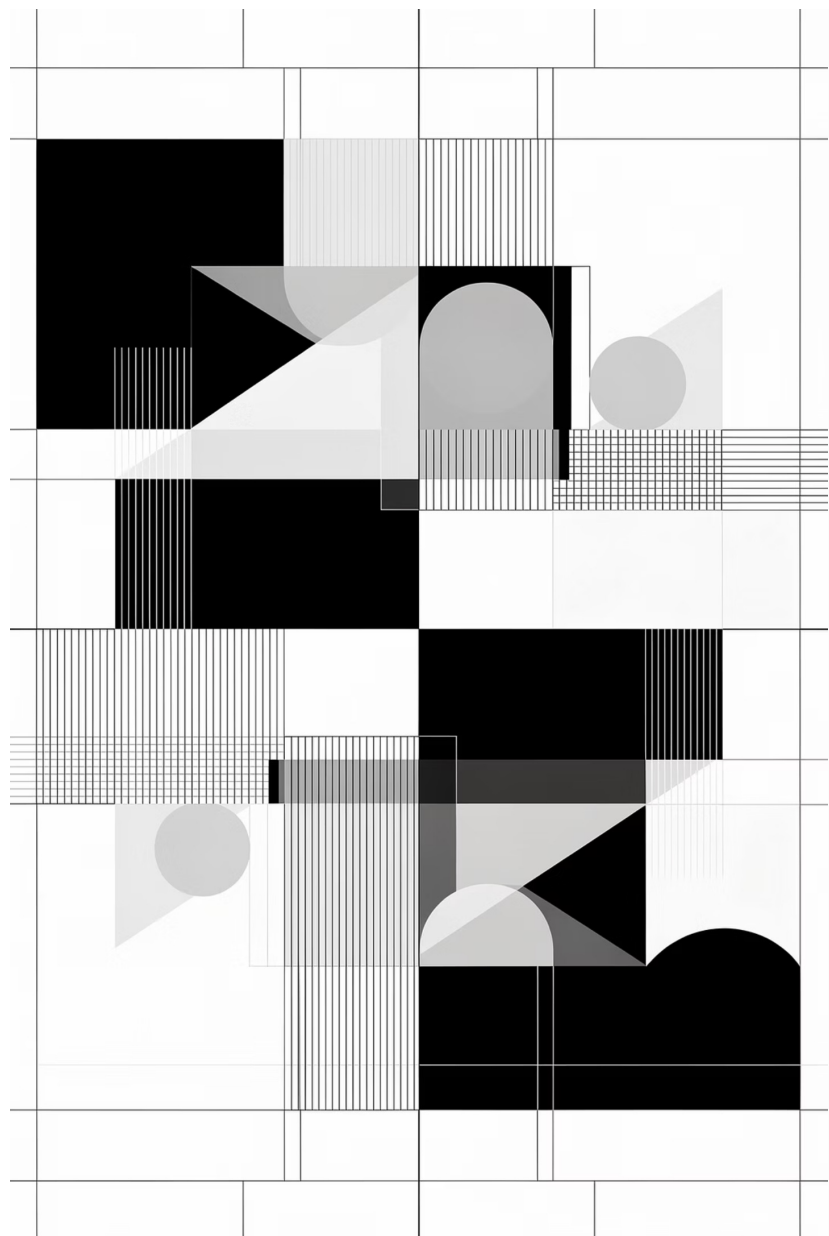


תמיכה נדירה

תמיכה בפונקציות מחלקה מבוססות תבנית עדיין נדירה בקומפיילרים רבים.



התשובה הסופית: לא ניתן לגרום למצביעים חכמים להתנהג בדיוק כמו רגילים בנושא המרות ירושה. אם יש צורך — ניתן לספק פונקציית המרה מתבנית ובמקומות של דו-משמעות לבצע casting.



מצביעים חכמים קבועים (const)

עבור מצביעים רגילים יש ארבע אפשרויות של `const`. כיצד משיגים את אותה גמישות עם מצביעים חכמים?

ארבעת המינים של const

מצביע חכם הוא אובייקט המכיל מצביע — לכן יש שני מקומות לשים: `const`

```
SmartPointer<CD> p;  
// non-const object, non-const pointer
```

```
SmartPointer<const CD> p;  
// const object, non-const pointer
```

```
const SmartPtr<CD> p = &goodCD;  
// non-const object, const pointer
```

```
const SmartPtr<const CD> p = &goodCD;  
// const object, const pointer
```



בעיה: הקומפיילר לא מאפשר השמה מ `SharedPtr<CD>` ל `SharedPtr<const CD>` — הם טיפוסים שונים מבחינתו, למרות שזה הגיוני.

הדימיון לירושה ציבורית

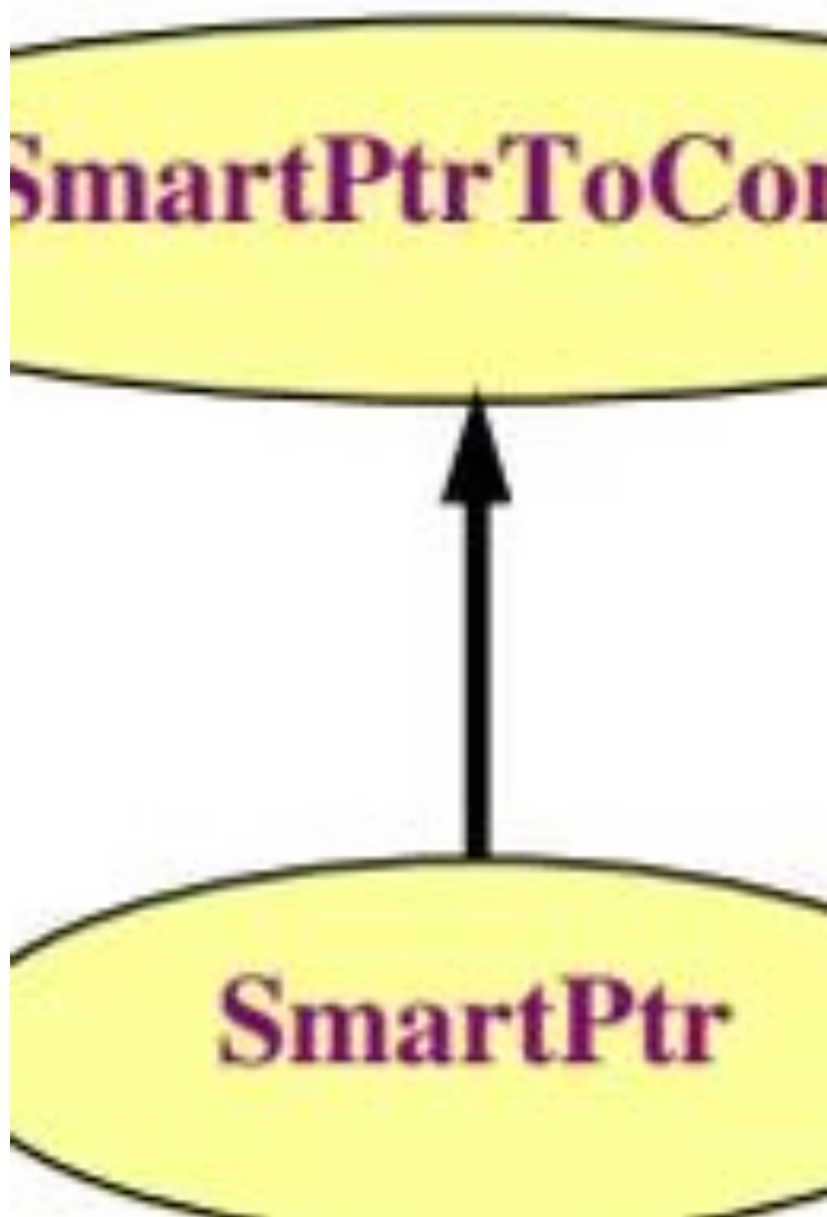
חוקי ירושה ציבורית

- ניתן להמיר אובייקט מהמחלקה הנגזרת לאובייקט ממחלקת הבסיס
- כל מה שניתן לבצע עם אובייקט הבסיס — ניתן גם עם הנגזרת. אך לא להיפך.

חוקי const

- ניתן להמיר מצביע non-const למצביע const
- כל מה שניתן לבצע עם const ניתן גם עם non-const אך לא להיפך.

את הדימיון הזה ניתן לנצל SmartPtr: יירש מ SmartPtrToConst-ובאמצעות union פנימי כל אחד יגש למצביע הנכון.



מימוש SmartPtrToConst

```
template<class T>
class SmartPtrToConst
{
    ...
protected:
    union
    {
        const T* constPointee; // for SmartPtrToConst access
        T* pointee;             // for SmartPtr access
    };
};

template<class T>
class SmartPtr : public SmartPtrToConst<T>
{
    ... // no data members
};
```

יתרון ה-union: כל אובייקט מכיל רק מצביע אחד מבחינת גודל. יש לממש בזהירות כדי שכל מחלקה תגש למצביע הנכון.



מילה לסיום

האם היתרונות בשימוש במצביעים חכמים שווים את הטרחה שהם גוררים?

יתרונות ✓

- מניעת דליפות זיכרון
- שליטה על בנייה, הריסה והעתקה
- שימושיים במיוחד במקרים ספציפיים

חסרונות ✗

- קשים יותר למימוש והבנה
- בעיות בהמרות למצביעים רגילים
- בדיקות ריקנות מוגבלות
- לא ניתן לממש מצביע חכם כללי לחלוטין